🟢 Aula 3 — Componentes e Props (4h)

🎯 Objetivos da aula

- Entender **componentização** como base de SPAs modernas.
- Criar **componentes reutilizáveis** no Vue.
- Utilizar **props com tipagem e validação**.
- Implementar **comunicação pai → filho**.
- Organizar o projeto seguindo **padrões de mercado**.

---

1) Teoria (aprox. 1h30)

1.1 Componentização

Componentes são **blocos independentes de UI + lógica**, que:

- Podem ser reutilizados em várias telas.
- Facilitam manutenção e testes.
- Representam responsabilidades únicas (SRP).

Exemplos comuns:

- `UserForm`
- `UserList`
- `Header`
- `Footer`

No Vue, cada componente é um **Single File Component (SFC)**:

```
Template + Script + Style
```

1.2 Props e Tipagem

Props são dados **recebidos do componente pai**.

Características:

- São **imutáveis** no componente filho.
- Podem (e devem) ser **tipadas**.
- Podem ser **obrigatórias ou opcionais**.

Exemplo conceitual:

```
props: { titulo: { type: String, required: true } }
```

1.3 Comunicação Pai → Filho

Fluxo padrão do Vue:

```
Pai → (props) → Filho
```

- O **pai** controla o estado.
- O **filho** apenas consome e exibe.
- Modificações de estado continuam no pai.

✔️ Padrão usado em sistemas profissionais.

---

2) Prática (aprox. 2h30) — Projeto organizado por componentes

> 🎯 **Contexto**
> Refatorar o mini cadastro de usuários (Aula 2) para:

- Componentes reutilizáveis
- Organização por pastas
- Comunicação clara pai → filho

2.1 Estrutura de pastas (padrão mercado)

Crie a seguinte estrutura em `src/`:

```
src/
 ├─ components/
 │   ├─ UserForm.vue
 │   ├─ UserList.vue
 │   └─ UserItem.vue
 ├─ App.vue
 └─ main.js
```

---

3) Código comentado — Componentes

3.1 `App.vue` — Componente Pai (orquestrador)

```
<template> <main class="container"> <h1>ILP3 – Vue.js Aula 3</h1> <!-- Componente filho recebendo função e dados --> <UserForm
@salvar="adicionarUsuario" /> <!-- Passagem de dados do pai para o filho via props --> <UserList :usuarios="usuarios" /> </main>
</template> <script> // Importa os componentes filhos import UserForm from './components/UserForm.vue' import UserList from
'./components/UserList.vue' export default { name: 'App', components: { UserForm, UserList }, data() { return { usuarios: [],
proximoId: 1 } }, methods: { adicionarUsuario(dadosUsuario) { // Recebe dados do filho e atualiza o estado this.usuarios.push({
```

```
id: this.proximoId++, ...dadosUsuario }) } } } </script> <style scoped> .container { max-width: 900px; margin: 40px auto; font-family: Arial, sans-serif; } </style>
```

3.2 `components/UserForm.vue` — Componente Filho (Formulário)

```
<template> <section class="card"> <h2>Cadastro de Usuário</h2> <input v-model="nome" type="text" placeholder="Nome" /> <input v-model="email" type="email" placeholder="Email" /> <button @click="salvar"> Salvar </button> </section> </template> <script> export default { name: 'UserForm', data() { return { nome: '', email: '' } }, methods: { salvar() { // Emite evento para o componente pai this.$emit('salvar', { nome: this.nome, email: this.email, ativo: true }) // Limpa o formulário this.nome = '' this.email = '' } } } </script> <style scoped> .card { border: 1px solid #ddd; padding: 20px; border-radius: 10px; margin-bottom: 20px; } input { display: block; width: 100%; margin-bottom: 10px; padding: 8px; } </style>
```

> ⚠️ Observação didática
> Aqui já aparece **comunicação filho → pai via evento**, preparando terreno para a Aula 4.

3.3 `components/UserList.vue` — Componente Filho (Lista)

```
<template> <section class="card"> <h2>Usuários</h2> <!-- Renderização condicional --> <p v-if="usuarios.length === 0"> Nenhum usuário cadastrado. </p> <ul v-else> <!-- Passa cada usuário como prop para outro componente --> <UserItem v-for="usuario in usuarios" :key="usuario.id" :usuario="usuario" /> </ul> </section> </template> <script> import UserItem from './UserItem.vue' export default { name: 'UserList', components: { UserItem }, // Props recebidas do pai (App.vue) props: { usuarios: { type: Array, required: true } } } </script> <style scoped> .card { border: 1px solid #ddd; padding: 20px; border-radius: 10px; } </style>
```

3.4 `components/UserItem.vue` — Componente Filho (Item)

```
<template> <li class="item"> <strong>{{ usuario.nome }}</strong> — {{ usuario.email }} <span class="status" :class="{ ativo: usuario.ativo }" > {{ usuario.ativo ? 'Ativo' : 'Inativo' }} </span> </li> </template> <script> export default { name: 'UserItem', // Tipagem e validação de props props: { usuario: { type: Object, required: true } } } </script> <style scoped> .item { margin-bottom: 8px; } .status { margin-left: 10px; padding: 2px 6px; border-radius: 6px; background: #ccc; font-size: 12px; } .status.ativo { background: #4caf50; color: white; } </style>
```

---

4) O que foi consolidado nesta aula

✔️ Componentização

- UI dividida em responsabilidades claras
- Código mais legível e escalável

✔️ Props com tipagem

```
props: { usuario: { type: Object, required: true } }
```

✔️ Comunicação Pai → Filho

```
<UserList :usuarios="usuarios" />
```

---

5) Atividade prática da aula

🎯 Desafio

1. Criar um componente `UserHeader.vue`
2. Receber via prop:
   - total de usuários
3. Exibir:
   - "Total de usuários cadastrados: X"
4. Usar o componente no `App.vue`

💡 Dica de prop:

```
props: { total: Number }
```

---

6) Checklist de encerramento

- Criou componentes reutilizáveis
- Usou props corretamente
- Organizou pastas
- Entendeu fluxo pai → filho
- Aplicou tipagem de props